

TP : Introduction au NLP avec Python et un Character-Level RNN

Objectif

L'objectif de ce TP est de mettre en pratique les bases du **Natural Language Processing (NLP)** en utilisant **Python** et **PyTorch**, à travers la réalisation d'un modèle simple de **character-level Recurrent Neural Network (RNN)**.

À l'issue de ce TP, vous devrez être capables de :

- manipuler des données textuelles en Python,
- transformer des mots en représentations numériques,
- implémenter et entraîner un modèle RNN simple,
- analyser les résultats obtenus.

Contexte

En **NLP**, un texte ne peut pas être utilisé directement par un modèle de deep learning. Il doit d'abord être **prétraité** (*preprocessing*).

Un **character-level RNN** traite un mot comme une séquence de caractères. À chaque caractère, le modèle met à jour un **hidden state** qui représente l'information mémorisée. À la fin du mot, le modèle produit une **prédiction** correspondant à une **classe**.

Dans ce TP, vous allez construire un modèle capable de prédire la langue d'origine d'un nom à partir de son orthographe.

Données

Le jeu de données est fourni sous forme de fichiers texte (`.txt`). Chaque fichier correspond à une langue et contient une liste de noms (un nom par ligne).

Chaque exemple du dataset est composé de :

- un nom (texte),
- une étiquette correspondant à la langue.

Travail demandé

Vous devez réaliser les étapes suivantes **en Python** :

1. Préparation de l'environnement

- Importer les bibliothèques nécessaires (`torch`, `numpy`, etc.).
- Vérifier la disponibilité du **CPU** ou de **CUDA**.

2. Preprocessing du texte

- Convertir les chaînes de caractères Unicode en caractères ASCII exploitables.
- Définir un vocabulaire de caractères autorisés.
- Gérer les caractères inconnus.

3. Représentation des données

- Transformer un caractère en vecteur numérique (*one-hot encoding*).
- Transformer un mot en une séquence de vecteurs.
- Vérifier les dimensions des tenseurs obtenus.

4. Construction du dataset

- Charger les données depuis les fichiers texte.
- Associer chaque nom à son label.
- Séparer les données en ensemble d'entraînement et de validation.

5. Modélisation

- Implémenter un **character-level RNN** avec **PyTorch**.
- Définir la fonction de perte (*loss function*) et l'optimiseur.
- Expliquer le rôle des différentes couches du modèle.

6. Entraînement

- Entraîner le modèle sur l'ensemble d'entraînement.
- Observer l'évolution de la **loss**.
- Ajuster les hyperparamètres si nécessaire.

7. Évaluation

- Tester le modèle sur l'ensemble de validation.
- Analyser les prédictions obtenues.
- Identifier les erreurs de classification les plus fréquentes.

Rendu attendu

Vous devez rendre :

- un **notebook Jupyter (.ipynb)** fonctionnel,
- des cellules Markdown expliquant les choix effectués,
- des visualisations simples (loss, résultats).

Le notebook doit être clair, structuré et commenté.

Consignes

- Le travail est **en binôme**.
- Le code doit être lisible et bien organisé.
- L'accent est mis sur la compréhension du processus, pas sur la performance finale.

Bonus (facultatif)

- Remplacer le **RNN** par un **LSTM** ou un **GRU**.
- Tester le modèle sur de nouveaux noms.
- Comparer les résultats entre plusieurs architectures.